

System Deployment and User Guide

This user guide provides detailed instructions for the deployment and use of the electric vehicle charging station management system. It is intended for the personnel responsible for the maintenance and operation of the system.

File Description

The directory containing the entire project is available on the GitHub repository at this link: [https://github.com/Gasile/TB_CSMS]. This repository is structured as follows:

- **.github**: Contains the GitHub Actions workflows for continuous integration.
- **citrineos-config**: Contains the configuration files for the CitrineOS CSMS, allowing the construction of the citrine package image. It also contains the database modification data corresponding to the additions made during development.
- **citrineos-core**: Contains the source code of the CitrineOS CSMS. This folder was cloned from the official CitrineOS repository. The only modifications made are located in the **citrineos-core/apps/Server/everest** folder. This folder contains the simulation environment for the 5 virtual Everest charging stations. Usage details for these simulators can be found in the section “Setting up the Simulators”.
- **frontend**: Contains the source code of the frontend developed in React. This folder also contains the configuration files for building the frontend package image.
- **services**: Contains the subfolders for each microservice developed in Go. Each subfolder contains the source code of the microservice as well as the configuration files for building the corresponding package image.

At the root of the project, there are also configuration files such as environment files, the .gitignore, and two versions of the docker-compose file.

The **docker-compose-deploy.yml** file is used for deployment using Docker images. This file operates independently. As specified in the report, the proper functioning of this version could not be ensured.

The **docker-compose.yml** file is also used for the complete launch of the project. However, this file requires a full clone of the repository and does not use the Docker images.

Launching the Project

Launching with Docker Images

1. Retrieve the **docker-compose-deploy.yml** file and the **.env.example** file from the GitHub repository.
2. Rename the **docker-compose-deploy.yml** file to **docker-compose.yml** and the **.env.example** file to **.env**, then fill in the environment variables.
3. Place the files in a working directory.
4. Open a terminal and navigate to this directory.
5. Pull the required Docker images by executing the following command:

```
docker-compose pull
```

6. Launch the project by executing the following command:

```
docker-compose up
```

Launching with the Complete Repository

This method is the most reliable. It is the one used throughout the development of this project. It is also the one currently running on the HEI server.

1. Clone the complete GitHub repository using the following command:

```
git clone https://github.com/Gasile/TB_CSMS
```

2. Navigate to the cloned directory:

```
cd TB_CSMS
```

3. Rename the **.env.example** file to **.env** and fill in the environment variables.
4. Launch the project by executing the following command:

```
docker-compose up
```

It may happen that the citrine container attempts to start before the containers it depends on, which can cause a failure. In this case, simply rerun the `docker-compose up` command until the container starts correctly.

System Usage

Once the system is launched, several web interfaces are available to perform various actions. These interfaces are accessible via a web browser using the server's IP address and the corresponding port for each interface.

Hasura GraphQL Engine

Access to the Hasura GraphQL Engine web interface is available on port 8090. This interface allows interaction with the database via GraphQL queries. This makes it possible to directly supervise the data tables to verify the proper functioning of the system. It is also possible to execute queries to retrieve specific data or to make modifications to the database. To access it, you must provide the `HASURA_GRAPHQL_ADMIN_SECRET` specified in the `.env` file during project launch.

CitrineOS Central System API

Access to the CitrineOS Central System API web interface is available on port 8080. This interface allows interaction with the CitrineOS CSMS via HTTP requests. This enables direct supervision of the charging stations to verify the proper functioning of the system. It is possible to send requests to the charging stations, allowing for actions such as restarting a station or requesting its startup message (BootNotification).

Frontend Dashboard

The frontend is accessible via the configurations established beforehand during its development. In its current state, it is accessible at the address `evse.hevs.ch`. This configuration is contained in the `docker-compose.yml` file. In the environment variables contained within the frontend image, it is possible to enable an option that activates user and administrator login shortcuts. This avoids having to enter an email and password at each connection. However, it is crucial not to activate this option if the deployment environment is exposed to the internet.

System Update

Microservices, Frontend, and CitrineOS CSMS

As previously mentioned, GitHub Actions have been configured to allow the automatic building of Docker images for each microservice, the frontend, and the CitrineOS CSMS. These actions are triggered every time a modification is pushed to the GitHub repository. This ensures that the Docker images are always up to date with the source code.

If the `docker-compose.yml` used is the version without the images, once the Go code of a microservice or the React code of the frontend is modified, simply run the following command:

```
docker compose up -d --build <container_name>
```

Database Modification

As explained above, the **`citrineos-config`** folder contains the database modification data according to what was added during development. These modifications are carried out via SQL scripts that are executed when the citrine container is launched. This ensures that the database is always up to date with the latest modifications made to the system.

To simplify database modification, the Hasura web interface provides tools to do this in a “graphical” way. In order to save the modifications made via this interface, it is possible to launch a “console” that will listen to and record the modifications made, and add them to the **`citrineos-config`** folder.

Before starting this procedure, you must clone the git repository to a local directory and add the `hasura.exe` executable to the `citrineos-config/hasura` folder. This executable can be found in the folder submitted for archiving, which contains all the project files. It can also be downloaded online.

1. Open an SSH tunnel using this command:

```
ssh -L 8090:localhost:8090 basile.gasser@vlenpchrman.hevs.ch
```

Use valid credentials that have SSH access permissions to the server here.

2. Open a terminal in the `TB_CSMS/citrineos-config/hasura` folder.
3. Run the following command:

```
./hasura.exe console --admin-secret <HASURA_GRAPHQL_ADMIN_SECRET>
```

Once the command is entered, a window should open automatically in a browser at `http://localhost:8090/console`

4. Make the desired modifications.

If the modifications concern table or column changes, it is important to check the “This is a migration” box.

5. In the terminal, stop the execution by pressing `Ctrl+C`. After this, the modifications will have been added locally to the `citrineos-config` folder.
6. Version and push the generated migration and metadata files.

```
git add citrineos-config/hasura/  
git commit -m "migration(db): "  
git push
```

Setting up the Simulators

In the citrineos-core/apps/Server/everest folder, there is another docker-compose.yml file. Simply run the following command:

```
docker compose up -d
```

Each simulator has 3 containers. To modify the number of simulators, simply comment out the 3 containers of each simulator that should be removed in the docker-compose.yml.

Adding an RFID Badge

It is possible to access the Node-RED interface of the simulators via the port indicated under the nodered-X container of a simulator. In the RFID tab on the is_token block, it is possible to add new badges to the “Options” list. The ID must absolutely consist of hexadecimal characters and be 8 or 16 characters in length, otherwise CitrineOS will not recognize the badge. Once the badges are added, go to the menu in the top right, then to Export. In the window that opens, below “Export,” select “all flows” and then the JSON format. Next, download the file and replace the shared-flow.json file in the everest folder with the newly downloaded file.

It is also possible to directly modify the badge list in the JSON file around line 1170.

Once this file has been modified, simply restart the nodered container of the simulators with this command:

```
docker compose restart nodered-1 nodered-2 nodered-3 nodered-4 nodered-5
```

Used port List

Port	Service	Description
80, 443	Reverse Proxy (Traefik)	External entry point of the system. Port 80 forces redirection to HTTPS.[cite: 1] Port 443 handles encrypted traffic and dynamically routes it to web interfaces and APIs based on the requested path.
8081	CitrineOS	Main server entry point for communication with external charging stations.
8080	CitrineOS	Port providing access to the internal CitrineOS management API, including the documentation interface accessible via the /docs route.
8090	GraphQL Engine (Hasura)	Direct mapping allowing access to the database engine from the host machine (for administration console or maintenance) without passing through the public reverse proxy.
8086	Auth Service	Restricted access to the host’s loopback interface (127.0.0.1), allowing secure internal debugging or administrative operations.

Table 1: Exposed ports on the remote server and their description